How the flood-cascade project works

From the simulator, to the fast model that learns from it, to how we score it

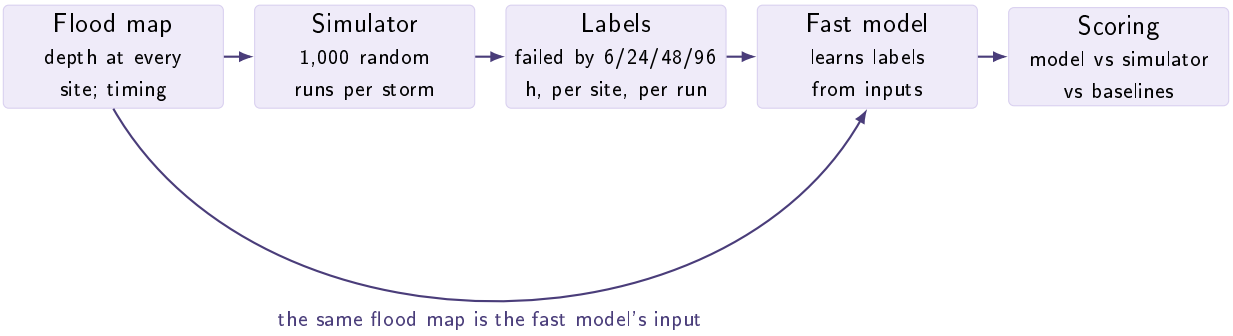
What this document is for. It walks through the whole project in the order things happen, in plain language, with the maths written out where it matters. It also answers the four points Dr. Lin asked for: every variable is defined where it first appears and collected again in Section 8; the problem (what is known, what we want to find) is stated in Section 3.1 for the simulator and Section 5.1 for the fast model; every step has its equation; and the performance metrics are in Section 7.

The whole project on one page

The question the project answers is: *if a coastal storm floods part of New York City, which pieces of infrastructure fail, when, and how many?* The flood itself only knocks out the sites that get wet. The larger damage comes afterwards, as failures spread through dependencies: a flooded substation takes down cell towers hours later, a dead pumping station backs up the water network, a hospital on generator power runs out of fuel after four days. That spreading is the *cascade*.

Two tools do the work:

- **The simulator** is a rule-based program. Give it a flood map and it plays the cascade forward, hour by hour, on a network of 6,231 sites. Because some rules involve chance (which flooded site actually fails, how long a backup lasts), it is run 1,000 times per storm and the results are averaged. One run takes 0.2 to 1.5 seconds; 1,000 runs take up to half an hour.
- **The fast model** is a neural network that learns to imitate the simulator. Given the same starting information it returns, for every site, the probability of having failed by 6, 24, 48 and 96 hours, in 11 milliseconds. It is trained on the simulator's runs. Its purpose is to make it possible to scan thousands of storms, which the simulator cannot do in reasonable time.



The rest of this document follows that arrow from left to right. Section 1 describes the network. Section 2 describes a storm. Sections 3 and 4 describe one simulator run and what 1,000 runs give.

Section 5 describes what the fast model is given and what it must return, Section 6 how it computes and learns, and Section 7 how we measure whether it is any good.

1 The city as a network

Sites. The city is represented by $N = 6{,}231$ sites, each belonging to one of six systems: 203 power substations, 4,150 cell-tower clusters, 61 hospitals, 493 subway stations, 137 water facilities (pumping stations and treatment plants) and 1,187 fuel sites (terminals and gas stations). We write V for the set of all sites, i or j for one site, $a(i)$ for its system, and V_a for all sites of system a (so $N_{\text{telecom}} = 4{,}150$). Each site has a short list of fixed attributes $\mathbf{x}_i$ (position and a few system-specific facts such as number of towers or bed count), 8 numbers scaled to the range 0–1.

Dependency links. The cascade travels along *dependency links*. A link (j, i) means "site i needs site j to keep working". There are 7,513 such links of four kinds: power supply (a substation feeds towers, stations, hospitals, pumps, fuel sites), fuel supply (a terminal or station supplies generators at hospitals, towers and pumps), water supply (a plant supplies a hospital) and SCADA monitoring (a cell cluster carries the control signal of a substation). Each link carries an engineering estimate $\bar{b}_{ji}$ of how many hours i can keep going after j fails: the *buffer*.

link	what it represents	median buffer $\bar{b}_{ji}$ (h)
power → subway	station loses power	0
power → fuel	pump at a gas station or terminal loses power	0 or 4
power → water	pumping station on backup	3
power → telecom	cell site on battery	6
power → hospital	hospital on generator	96
telecom → power (SCADA)	substation loses remote control	2
water → hospital	hospital's water storage	24
fuel → telecom, fuel → water	generator fuel reserve	48
fuel → hospital	generator fuel reserve	96

Other links. The network file also holds within-system links (power lines, subway lines, water flow, fuel distribution), repair-access links from subway stations, and 23,646 telecom "failover" links between neighbouring cell sites of the same carrier. The simulator uses the within-system links inside its within-system rules (Step 4 of Section 3). The fast model sees all of them: 37,355 directed links in 20 relation types, written E.

The maths

V: all sites, $|V| = N = 6{,}231$. $a(i) \in \{\text{power, telecom, hospital, subway, water, fuel}\}$: system of site i. V_a, N_a: sites of system a and their number. $\mathbf{x}_i \in [0,1]^8$: fixed attributes. E^{dep}: dependency links (j, i), "i needs j", with median buffer $\bar{b}_{ji} \geq 0$ hours. E: all links, split by relation type ρ into E_ρ, 20 types.

2 A storm: what we know before anything fails

A storm enters the project as a *flood map* from a flood model (Gwyn's GISSR model for most storms; GeoClaw for three). For storm s it gives every site a peak flood depth d_i^s in metres. Sites

with $d_i^s > 0$ are the *wet sites*, W^s. From the storm's hydrograph we also know when the water level peaks, t_{pk}^s hours after the storm starts, and for every wet site the hour the water arrives, α_i^s, how many hours it stays wet, u_i^s, and the *lead* ℓ_i^s: how many hours before the peak the site gets wet. A "fast" storm has leads of about 2 hours; a "slow" one about 12.

This week the project has 70 storms: 22 from earlier work (19 GISSR storms of varied size and 3 GeoClaw maps) and 48 new GISSR storms chosen by a rule written in advance (40 for training, 8 kept unseen).

The maths

Known for storm s: $d_i^s \geq 0$ for all i; $W^s = \{i : d_i^s > 0\}$; t_{pk}^s; and for $i \in W^s$ the hours α_i^s, u_i^s, ℓ_i^s (with $\ell_i^s = t_{\text{pk}}^s - \alpha_i^s$).

3 One simulator run, step by step

3.1 What the simulator is asked

Known. The network (V, E^{dep}) with its median buffers $\bar{b}$ and the fragility and within-system parameters; for a storm, the depth map d^s and the timing $(t_{\text{pk}}^s, \alpha^s, u^s)$.
Unknown. The failure time of every site, $\mathbf{T} = (T_i)_{i \in V}$, which is random. What we want is its distribution, summarised as $\pi_{ik}^s = \Pr(T_i \leq t_k \mid s)$ for every site and horizon, and the distribution of the cascade size. There is no formula for these; the simulator estimates them by sampling, as the five steps below describe.

3.2 A run in five steps

A run is one possible way the storm could play out. The simulator makes two kinds of random draw at the start of a run, then plays the cascade forward deterministically.

Step 1. Which sites does the water knock out directly?

Not every wet site fails. A subway station in 10 cm of water is in real trouble; a substation in 10 cm of water is probably fine. So each wet site fails with a probability that rises with depth, following a *fragility curve* of the standard Hazus form: a lognormal curve with a median depth δ_a (the depth at which half the sites of that system fail) and a spread β_a. The simulator then flips a biased coin for each site. The sites that come up "failed" are the *seeds* of the run, written $m_i = 1$. The 99 sites outside the city footprint never seed.

The maths

$$\phi_i^s = \Phi\left(\frac{\ln d_i^s - \ln \delta_{a(i)}}{\beta_{a(i)}}\right) \quad \text{if } d_i^s > 0, \qquad \phi_i^s = 0 \text{ otherwise;}$$

$$m_i \sim \text{Bernoulli}(\phi_i^s) \quad \text{independently for each site.}$$

Φ is the standard normal distribution function (the S-shaped curve from 0 to 1). Medians δ_a in metres: power 0.6, telecom 0.3, hospital 0.6, subway 0.1, water 0.5, fuel 0.5. Spreads β_a: 0.5, 0.6, 0.5, 0.4, 0.5, 0.6. The number of seeds in a run is $n(\mathbf{m}) = \sum_i m_i$.

Worked example

Three sites, all in 0.3 m of water. Subway station: $(\ln 0.3 - \ln 0.1)/0.4 = 2.75$, so $\Phi(2.75) = 0.997$: it almost surely fails. Cell cluster: $(\ln 0.3 - \ln 0.3)/0.6 = 0$, so $\Phi(0) = 0.5$: a coin flip. Substation: $(\ln 0.3 - \ln 0.6)/0.5 = -1.39$, so $\Phi(-1.39) = 0.08$: it usually survives. At 1 m the substation's probability rises to 0.85.

Step 2. How long does each backup last in this run?

The buffers in the table of Section 1 are medians. In a real event one hospital's generator runs for three days and another's for six. So in every run each buffer is drawn afresh from a Weibull distribution whose median is the engineering value. The shape of the distribution (how spread out it is) is set per receiving system. Links with a zero buffer stay at zero: a subway station with no power stops immediately.

The maths

$$B_{ji} \sim \text{Weibull}(\text{shape } \kappa_{a(i)},\ \text{scale } \lambda_{ji}), \qquad \lambda_{ji} = \frac{\bar{b}_{ji}}{(\ln 2)^{1/\kappa_{a(i)}}},$$

which makes the median of B_{ji} exactly $\bar{b}_{ji}$. Shapes κ: hospital 2.5, water 2.5, telecom 3.0, power 3.0, fuel 2.0, subway 4.0. Buffers are drawn independently of each other.

Worked example

Power → hospital, median 96 h, shape 2.5: in 10 % of runs the generator lasts under 45 h, in 10 % over 155 h. Power → telecom, median 6 h, shape 3.0: between 3.2 h and 9.0 h in 80 % of runs.

Step 3. When do the seeds fail? (the two label types)

The project uses two conventions, and trains a separate fast model for each.

- **Static labels.** All seeds fail at the same moment, hour 0, as if the whole flood arrived at once. The cascade is then evaluated at hours 6, 24, 48 and 96.
- **Arrival labels** (also called dynamic). Each seed fails at the hour the water actually reaches it, α_i^s. The cascade is evaluated every 6 hours up to the peak, then at the peak plus 0, 6, 24, 48, 96, 144, 240 and 360 hours. In this convention the clock for the labels is the storm's peak: "failed by 24 h" means by 24 hours after the peak.

Under arrival labels the cascade gets a head start on sites that flood early, so cascades are larger (2–50 % more failures than static labels on the same storm), and a slow storm ends with more failures than a fast one of the same size.

Step 4. How failures spread

Time moves along the evaluation grid. At each grid hour g the simulator applies two rules, over and over, until neither adds a new failure. Failure is permanent: there is no repair in the model.

Rule A, dependencies. A working site fails when one of its suppliers has been dead for longer than the buffer drawn for that link.

The maths

Let T_i be the hour site i fails ($T_i = \infty$ if it never does). At grid hour g:

$$\text{if } i \text{ is still working and some supplier } j \text{ of } i \text{ has } T_j + B_{ji} \le g, \quad \text{then } T_i \leftarrow g.$$

Rule B, within a system. After Rule A, each system checks whether the sites it has lost cause further losses inside the same system:

- *Subway (overload).* Each station carries a load equal to how many shortest paths pass through it. When stations close, paths reroute and loads shift. A station fails when its new load exceeds its capacity, set at 1.2× its normal load (the Motter–Lai rule). This repeats until no station is over capacity.
- *Water and fuel (flow).* If a water treatment plant fails, every pumping station upstream of it fails (there is nowhere to send the water). If a fuel terminal fails, every station it supplies fails.
- *Telecom (capacity and spill).* When a cell site dies, its users move to neighbouring sites of the same carrier within 3 km. Each site can serve up to a capacity L_i^0/ρ, with $\rho = 0.6$ its normal busy-hour utilisation; demand it cannot take spills on, for at most two hops. On top of this, demand everywhere is multiplied by a surge $\mu(t) = 1 + 0.5\,e^{-t/12}$ (phones are busiest right after the event). A site crashes only if the demand offered to it exceeds three times its capacity.
- *Hospital (patient transfer).* Patients of a failed hospital are sent to open hospitals within 40 km, with a preference for nearby ones and for hospitals in the same health system. A hospital stops accepting at 95 % of its beds and fails if its census passes 120 %.
- *Power.* No within-system rule in these campaigns; substations fail by flood or by losing their SCADA link.

Rules A and B feed each other: a subway closure can cut repair access, a crashed cell cluster can take a substation's control signal, and so on, all at the same grid hour.

Step 5. What one run gives

At the end of a run every site has a failure time T_i. From it we read, for any hour t, whether the site had failed by then. Counting the sites gives the cascade's size at 6, 24, 48 and 96 hours.

4 From 1,000 runs to probabilities and labels

4.1 Why 1,000 runs

One run is one roll of the dice. To say how *likely* a site is to fail, we repeat the run 1,000 times with fresh coin flips and fresh buffers. The failure probability of a site at a horizon is simply the fraction of runs in which it had failed by then. The simulator never outputs a probability directly; probabilities are frequencies over runs.

The maths

With $R = 1{,}000$ runs $r = 1, \ldots, R$ of storm s and horizons $t_1, t_2, t_3, t_4 = 6, 24, 48, 96$ h:

$$\pi^s_{ik} = \frac{1}{R} \sum_{r=1}^{R} \mathbb{1}\left[T_i^{(r)} \le t_k\right]$$

is the simulator's failure probability of site i by horizon t_k. The average cascade size at t_k is $\frac{1}{R}\sum_r \sum_i \mathbb{1}[T_i^{(r)} \le t_k]$.

4.2 Labels for the fast model

Each run becomes one training example. Its label is a table of zeros and ones: for every site and every horizon, did the site fail by then? Static labels count hours from 0; arrival labels count hours from the peak. Because failure is permanent, a row can only go from 0 to 1, never back.

The maths

$$y_{ik} = \mathbb{1}[\,T_i \le t_k\,] \ \text{(static)}, \qquad y_{ik} = \mathbb{1}[\,T_i - t^s_{\text{pk}} \le t_k\,] \ \text{(arrival)}, \qquad y_{i1} \le y_{i2} \le y_{i3} \le y_{i4}.$$

Hour 0 is not a horizon: at hour 0 the answer is just the seed list, which the model is given.

4.3 Which storms train, which storms test

Runs from the *training storms* are shuffled once and split 80/20 into a training set (used to fit the model) and a validation set (used to pick the best epoch and to report numbers). The *held-out storms* are never seen in training; they are the real test of whether the model generalises.

	training storms	held-out storms	total
earlier storms	17	5	22
new GISSR storms	40	8	48
all	57	13	70

With the 17 earlier training storms: 17,000 runs, of which 13,600 train and 3,400 validate. The runs of the 5 held-out storms (5,000) are scored separately, storm by storm.

5 The fast model: what it is given, what it must return

5.1 The learning problem, stated

The fast model is asked a slightly different question from the simulator. The simulator starts from the depth map and draws the seeds itself. The fast model is handed the seeds of a particular run and asked: *given that exactly these sites were knocked out, how likely is every other site to fail by each horizon?* The only randomness left to average over is the buffers (and, in the within-system rules, nothing: those are deterministic given the failed set).

Known (inputs) for one run. The network (V, E) and attributes $\mathbf{X}$; the seed indicator $\mathbf{m} = (m_i)$; and, in the models that use them, the timing inputs $\mathbf{Q}^s$ and the storm summary $\mathbf{c}$ (Section 5.2). All of these are known before the cascade starts.
Unknown (outputs). For every site and horizon the probability

$$p^\star_{ik} = \Pr(Y_{ik} = 1 \mid \mathbf{m},\ s)\,,$$

and from it two derived answers: the expected number of failed sites, $\sum_i p^\star_{ik}$, and each site's most likely failure window.
Task. Find parameters θ of a network f_θ such that $f_\theta(\text{inputs})_{ik} \approx p^\star_{ik}$, including on storms the model never trained on.

Seeds themselves are not a prediction problem: a seed has $y_{ik} = 1$ at every horizon and the model is told which sites are seeds. So everything is judged on the *non-seed* sites, $U = \{i : m_i = 0\}$.

5.2 The input of one site

Every site gets one input vector $\mathbf{z}_i$, assembled from up to four pieces:

1. **Fixed attributes $\mathbf{x}_i$**, 8 numbers (Section 1). One honest note: the elevation and flood-depth slots in the current network file hold placeholder constants, so the flood reaches the model only through the next three pieces.
2. **The storm summary $\mathbf{c}$** (added this week; only in the models with the storm-size input). Seven numbers, the same for every site: for each of the six systems, how much of it the flood knocked out, and the same for the city as a whole. Models with timing inputs get three more: the average arrival hour, average hours wet and average lead over the wet sites.
3. **The seed bit m_i**: 1 if the flood knocked out this site.
4. **Timing $\mathbf{q}^s_i$** (only in the models with timing inputs): the site's own arrival hour, hours wet and lead, each divided by 96 hours.

The maths

$$c_a = \frac{\ln\big(1 + n_a(\mathbf{m})\big)}{\ln(1 + N_a)} \quad (a = \text{power}, \ldots, \text{fuel}), \qquad c_{\text{all}} = \frac{\ln\big(1 + n(\mathbf{m})\big)}{\ln(1 + N)},$$

and $\bar{\mathbf{q}}^s$ is the mean of $\mathbf{q}_i^s$ over the wet sites, where $n_a(\mathbf{m})$ is the number of seeds in system a. Each c lies between 0 (nothing flooded) and 1 (everything flooded). The logarithm makes the number grow steadily with storm size instead of jumping.

$$\mathbf{q}_i^s = \tfrac{1}{96}\,(\alpha_i^s,\ u_i^s,\ \ell_i^s) \ \text{ for wet sites, } \mathbf{0} \text{ otherwise;} \qquad \mathbf{z}_i = \big[\ \mathbf{x}_i \parallel \mathbf{c} \parallel m_i \parallel \mathbf{q}_i^s\ \big],$$

with $\parallel$ meaning "placed side by side". Depending on which pieces a model uses, $\mathbf{z}_i$ has 9, 12, 16 or 22 numbers.

Worked example

Telecom has $N_a = 4{,}150$ sites. A small storm that floods 10 cell clusters gives $c_{\text{telecom}} = \ln 11 / \ln 4151 = 0.29$; 100 clusters give 0.55; 500 give 0.75. Every site in the city receives these same numbers, so even a tower in the Bronx with nothing flooded near it "knows" the storm is large.

Nothing in $\mathbf{c}$ or $\mathbf{q}$ comes from the cascade outcome. They are computed from the flood map and the seed list only, so the model is not being shown the answer.

6 How the fast model computes and learns

6.1 The computation, in four moves

The model is a graph neural network: it does the same small computation at every site, and lets sites exchange information along links. Its size is 64 numbers per site (the "hidden state"), two rounds of message passing, four attention heads. All weight matrices $\mathbf{W}$ and bias vectors $\mathbf{b}$ below are the trainable parameters θ; there are about 696,000 of them.

Move 1: each site summarises its own input. The input vector is multiplied by a matrix (one per system), a bias is added, and negative entries are set to zero. The result is the site's starting state, 64 numbers.

The maths

$$\mathbf{h}_i^{(0)} = \text{ReLU}\Big(\mathbf{W}_{a(i)}^{\text{in}}\mathbf{z}_i + \mathbf{b}_{a(i)}^{\text{in}}\Big), \qquad \text{ReLU}(x) = \max(x, 0).$$

Move 2: sites listen to their neighbours (twice). In each round, every site looks at the sites linked to it, separately for each relation type (its power supplier, its fuel supplier, its failover partners, ...). It does not treat all neighbours equally: it computes an *attention weight* for each neighbour by comparing a "query" made from its own state with a "key" made from the neighbour's state; the weights are normalised to sum to one. It then takes the weighted sum of the neighbours' "values", adds a transformed copy of its own state, sums this over all relation types, adds its previous state (a residual connection, so information is never lost), and applies ReLU. Four attention heads do this in parallel with different matrices and their results are concatenated. After two rounds, every site's state reflects what is within two links of it.

The maths

For round $l = 1, 2$, relation ρ, head c, and $\mathcal{N}_\rho(i)$ the sites linked to i by relation ρ:

$$\alpha_{ij,\rho}^{(l,c)} = \underset{j \in \mathcal{N}_\rho(i)}{\mathrm{softmax}} \frac{\big(\mathbf{W}_{Q,\rho}^{(l,c)} \mathbf{h}_i^{(l-1)}\big)^\top \big(\mathbf{W}_{K,\rho}^{(l,c)} \mathbf{h}_j^{(l-1)}\big)}{\sqrt{16}} \qquad \text{(attention weights, sum to 1 over } j\text{)}$$

$$\mathbf{g}_{i,\rho}^{(l)} = \Big\|_{c=1}^{4} \sum_{j \in \mathcal{N}_\rho(i)} \alpha_{ij,\rho}^{(l,c)} \, \mathbf{W}_{V,\rho}^{(l,c)} \mathbf{h}_j^{(l-1)} \; + \; \mathbf{W}_{S,\rho}^{(l)} \mathbf{h}_i^{(l-1)} \qquad \text{(message from relation } \rho\text{)}$$

$$\mathbf{h}_i^{(l)} = \mathrm{ReLU}\Big(\sum_{\rho \text{ ending at } a(i)} \mathbf{g}_{i,\rho}^{(l)} + \mathbf{h}_i^{(l-1)} \Big) \qquad \text{(new state)}$$

Why this created the problem we found this week. After two rounds, a site's state depends only on inputs within two links of it. Before the storm summary existed, a site with no flooded site within two links received exactly the same inputs in a small storm and a huge one, so the model had to give it the same answer: an average. That is why the model over-counted small storms by 3–4 times and under-counted big ones by a third. Giving every site the storm summary $\mathbf{c}$ removes the limit without adding rounds.

Move 3: each site turns its final state into four raw numbers. A small two-layer network (one per system) maps the 64-number state to 4 numbers $\eta_{i1}, \ldots, \eta_{i4}$, one per horizon.

The maths

$$\boldsymbol{\eta}_i = \mathbf{W}_{a(i)}^{(2)} \, \mathrm{ReLU}\Big(\mathbf{W}_{a(i)}^{(1)} \mathbf{h}_i^{(2)} + \mathbf{b}_{a(i)}^{(1)} \Big) + \mathbf{b}_{a(i)}^{(2)}.$$

Move 4: raw numbers become probabilities that can only go up. "Failed by 48 h" can never be less likely than "failed by 24 h". To guarantee this, the four raw numbers are read as *hazards*: h_{ik} is the chance the site fails *during* window k given it survived the earlier windows. The chance of surviving all windows up to k is the product of $(1 - h)$; one minus that is the probability of having failed by t_k.

The maths

$$h_{ik} = \sigma(\eta_{ik}) = \frac{1}{1 + e^{-\eta_{ik}}}, \qquad S_{ik} = \prod_{j \le k} (1 - h_{ij}), \qquad p_{ik} = 1 - S_{ik}.$$

Worked example

Raw outputs $\boldsymbol{\eta}_i = (-2, -1, 0, 1)$. Hazards: $h = (0.12,\ 0.27,\ 0.50,\ 0.73)$. Survival: $S = (0.88,\ 0.64,\ 0.32,\ 0.09)$. Probabilities of having failed by 6, 24, 48, 96 h: $p = (0.12,\ 0.36,\ 0.68,\ 0.91)$. They rise, as they must.

6.2 How the parameters are learned

Training means adjusting the 696,000 parameters so that the model's probabilities match the simulator's labels. The yardstick is the *log loss* (binary cross-entropy): for one site and horizon it is $-\ln p$ if the site failed and $-\ln(1-p)$ if it did not. It is small when the model gives high probability to what happened and very large when the model is confidently wrong.

Worked example

The site failed ($y = 1$). Model says $p = 0.9$: loss $-\ln 0.9 = 0.11$. Model says $p = 0.5$: loss 0.69. Model says $p = 0.1$: loss 2.30. Being sure and wrong costs twenty times more than being sure and right.

The loss of a run is the average over all 6,231 sites and 4 horizons. Training goes through the 13,600 training runs one at a time: compute the loss, compute how each parameter should move to reduce it (the gradient, by back-propagation), and take a small step in that direction with the Adam optimiser. Six passes over the data are made. After each pass the loss on the 3,400 validation runs is measured, and the version of the model with the lowest validation loss is kept.

The maths

$$\mathrm{bce}(p, y) = -\,y \ln p - (1 - y) \ln(1 - p), \qquad \mathcal{L}(\theta; \mathrm{run}) = \frac{1}{N \cdot 4} \sum_{i \in V} \sum_{k=1}^{4} \mathrm{bce}\left(p_{ik}, y_{ik}\right),$$

$$\hat{\theta} = \arg\min_{\theta} \frac{1}{|\mathcal{D}_{\mathrm{tr}}|} \sum_{\mathrm{runs} \in \mathcal{D}_{\mathrm{tr}}} \mathcal{L}(\theta; \mathrm{run}).$$

Adam, learning rate 10^{-3}, weight decay 10^{-5}, dropout 0.1, 6 epochs. The initial parameters are random; this week each setting was trained from two different random starts (PyTorch seeds 1 and 2) so that we can see how much results move between starts.

6.3 The baselines: three predictors with no neural network

To know whether the model has learned anything worth having, it is compared with three simple predictors built from the same training runs. Each knows a little more than the last.

1. **Base rate.** Every non-seed site gets the same number: the overall fraction of non-seed sites that fail by that horizon in the training runs.
2. **Site frequency.** Each site gets its own failure frequency over all training runs, ignoring which storm it is.
3. **Lookup table.** For each training storm, record each site's failure frequency in that storm's runs. For a new run, count its seeds, find the training storm with the closest average number of seeds, and reuse that storm's frequencies. This predictor knows the size of the storm and nothing else.

The maths

With n_i the number of training runs in which site i is not a seed, $\mathcal{D}_{\text{tr}}^s$ the training runs of storm s and n_i^s defined likewise:

$$\bar{f}_k = \frac{\sum_{\text{runs}} \sum_i (1 - m_i)\, y_{ik}}{\sum_{\text{runs}} \sum_i (1 - m_i)}, \qquad f_{ik} = \frac{\sum_{\text{runs}} (1 - m_i)\, y_{ik} + 0.5}{n_i + 1},$$

$$f_{ik}^s = \frac{\sum_{\text{runs} \in \mathcal{D}_{\text{tr}}^s} (1 - m_i)\, y_{ik} + f_{ik}}{n_i^s + 1}.$$

The lookup predicts $p_{ik} = f_{ik}^{s^\star}$ with $s^\star$ the training storm whose mean seed count is closest to $n(\mathbf{m})$. The small additions ($+0.5$, $+f_{ik}$) keep frequencies away from exactly 0 or 1.

7 How we score and compare models

Every metric is computed on non-seed sites, run by run, then averaged over the runs being scored: either the 3,400 validation runs or the 1,000 runs of one held-out storm. The same code scores the model and each baseline, so they are always compared on identical runs.

7.1 Per-site probability error: log loss

The same quantity as the training loss, but restricted to non-seed sites and reported per horizon and averaged over the four. Lower is better; 0 would be perfect; always answering "50 %" scores $\ln 2 = 0.69$.

The maths

$$\text{BCE}_k = \text{mean over runs of } \frac{1}{|U|} \sum_{i \in U} \text{bce}(p_{ik}, y_{ik}), \qquad \text{BCE} = \tfrac{1}{4} \sum_{k=1}^{4} \text{BCE}_k.$$

The Brier score is the same with $(p - y)^2$ in place of bce.

7.2 Does the model get the size of the cascade right? Count bias and count error

The model outputs probabilities, not a count. The count it implies is the *expected* number of failed sites, which is the sum of the probabilities: if 100 sites each have a 30 % chance, we expect 30 failures. There is no cut-off at 50 %. The simulated count is the number of non-seed sites that actually failed in the run.

Two numbers are reported. *Bias* asks whether the model is too high or too low on average; it can hide errors that cancel between runs. *Count error* (WAPE) adds up the size of every miss regardless of sign, so it cannot be fooled by cancellation.

The maths

Per run: $\hat{C}_k = \sum_{i \in U} p_{ik}$ (predicted), $C_k = \sum_{i \in U} y_{ik}$ (simulated). Over the runs scored:

$$\text{Bias}_k = \frac{\sum_{\text{runs}} (\hat{C}_k - C_k)}{\sum_{\text{runs}} C_k}, \qquad \text{WAPE}_k = \frac{\sum_{\text{runs}} |\hat{C}_k - C_k|}{\sum_{\text{runs}} C_k}.$$

"Predicted ÷ simulated" on the slides is $1 + \text{Bias}_k$ (1.0 is correct). Usually reported at 96 h.

Worked example

Three sites with probabilities 0.8, 0.5 and 0.1 contribute 1.4 expected failures. A run where the simulator has 200 non-seed failures and the model's probabilities add to 290 has a miss of 90; if every run looked like that, bias and count error would both be $90/200 = 45\,\%$. If half the runs were $+90$ and half -90, bias would be 0 but count error still 45 %.

7.3 Does it rank the right sites highest? AP and AUC

These two ignore the level of the probabilities and look only at their *order*. Sort the non-seed sites from most to least likely to fail. *Average precision* (AP) walks down that list and, each time it reaches a site that really failed, notes what fraction of the sites so far really failed; AP is the average of those fractions, and equals 1 when every failing site is ranked above every surviving one. *AUC* is the probability that a randomly chosen failing site is ranked above a randomly chosen surviving site. A model can have a high AP and still get the count badly wrong, which is exactly what happened before the storm summary.

The maths

With P_n, R_n the precision and recall after the n-th distinct score ($R_0 = 0$), n_+ failing and n_- surviving sites, and ties given their average rank:

$$\text{AP} = \sum_n (R_n - R_{n-1})\, P_n, \qquad \text{AUC} = \frac{\sum_{i:\, y_i = 1} \text{rank}(i) - n_+(n_+ + 1)/2}{n_+\, n_-}.$$

Computed per run and horizon, then averaged.

7.4 Yes/no flags

If a planner needs a list, sites with $p_{ik} \geq 0.5$ are flagged. Precision is the share of flagged sites that fail; recall is the share of failing sites that were flagged. Both are pooled over runs.

7.5 Does it get the timing right? Failure windows

The four cumulative probabilities define five windows: by 6 h, 6–24 h, 24–48 h, 48–96 h, and "not by 96 h". The probability mass in each window is the difference between consecutive cumulative probabilities. Among the sites that really fail, we ask whether the window with the most mass is the true one.

The maths

$$w_{i1} = p_{i1}, \quad w_{ij} = p_{ij} - p_{i,j-1}\ (j = 2,3,4), \quad w_{i5} = 1 - p_{i4}; \qquad \text{true window } j_i^\star = 5 - \sum_k y_{ik};$$

$$\mathrm{Acc}_{\mathrm{win}} = \frac{\#\{\text{failing non-seed sites with } \arg\max_{j\le 4} w_{ij} = j_i^\star\}}{\#\{\text{failing non-seed sites}\}}.$$

Worked example

From the earlier example, $p = (0.12, 0.36, 0.68, 0.91)$ gives window masses (0.12, 0.24, 0.32, 0.23, 0.09): the model's best guess is that the site fails between 24 and 48 hours.

7.6 Are the probabilities honest? Calibration

Take all the site-run pairs where the model said "about 30 %". If about 30 % of them really failed, the model is calibrated. The expected calibration error (ECE) sorts the pairs into ten probability bins and averages the gap between the stated probability and the observed failure rate.

The maths

$$\mathrm{ECE}_k = \frac{1}{n}\sum_{b=1}^{10}\Big|\sum_{(i,\mathrm{run})\in\mathcal{B}_b}(p_{ik} - y_{ik})\Big|,$$

n being the number of pairs and $\mathcal{B}_b$ the pairs whose p_{ik} falls in bin b.

7.7 Speed

Wall time of one forward pass of the model (all 6,231 sites, four horizons) against the wall time of one simulator run: 11 milliseconds against 0.2–1.5 seconds.

7.8 How comparisons are decided

- The model *beats* a baseline on a set of runs if its log loss is lower there.
- With two random starts per setting, one setting is *clearly* better than another only if both of its runs beat both of the other's.
- Validation runs come from the training storms, so a model can do well there by recognising the storm. Generalisation is judged on held-out storms only.
- Every experiment's pass/fail thresholds are written into the scoring script before the run (for example: "validation count error at 96 h must fall to 0.30 or below"). The script prints PASS or MISS; misses are reported as such.

8 Every symbol in one place

Symbol	Meaning (where it first appears)
$V,\ N,\ i,\ j$	all sites, their number (6,231), and indices for single sites (1)
$a(i),\ V_a,\ N_a$	system of site i; sites and number of sites of system a (1)
$\mathbf{x}_i$	8 fixed attributes of site i, scaled to $[0,1]$ (1)
$E^{\text{dep}},\ \bar{b}_{ji}$	dependency links "i needs j" and their median buffers in hours (1)
$E,\ E_\rho,\ \rho$	all 37,355 links, the links of relation type ρ, 20 types (1)
$s,\ d_i^s,\ W^s$	a storm, its peak depth at site i (m), its wet sites (2)
$t_{\text{pk}}^s,\ \alpha_i^s,\ u_i^s,\ \ell_i^s$	peak hour; arrival hour, hours wet and lead of wet site i (2)
$\delta_a,\ \beta_a,\ \Phi$	fragility median depth and spread for system a; normal distribution function (3, step 1)
$\phi_i^s,\ m_i,\ n(\mathbf{m}),\ n_a(\mathbf{m})$	direct-failure probability; seed indicator; number of seeds, overall and in system a (3, step 1)
$B_{ji},\ \kappa_a,\ \lambda_{ji}$	sampled buffer; Weibull shape for receiving system a; Weibull scale (3, step 2)
$g,\ T_i$	a grid hour; failure time of site i (3, step 4)
$R,\ r,\ \pi_{ik}^s$	number of runs (1,000), a run, the simulator's failure probability (4.1)
$t_k,\ k,\ y_{ik}$	horizons 6, 24, 48, 96 h; label "site i failed by t_k" (4.2)
$\mathcal{D}_{\text{tr}},\ \mathcal{D}_{\text{val}}$	training and validation runs (4.3)
U	non-seed sites of a run (5.1)
$p_{ik}^\star,\ p_{ik}$	target probability; the model's prediction (5.1)
$\mathbf{c},\ c_a,\ c_{\text{all}},\ \bar{\mathbf{q}}^s$	storm summary and its parts (5.2)
$\mathbf{q}_i^s,\ \mathbf{z}_i$	timing input of site i; full input vector of site i (5.2)
$\theta,\ \mathbf{W},\ \mathbf{b}$	trainable parameters: weight matrices and biases (6.1)
$\mathbf{h}_i^{(l)},\ \alpha_{ij,\rho}^{(l,c)},\ \mathbf{g}_{i,\rho}^{(l)}$	state of site i after round l; attention weight; message from relation ρ (6.1)
$\boldsymbol{\eta}_i,\ h_{ik},\ S_{ik},\ \sigma$	raw outputs; hazard; survival; logistic function (6.1)
bce, $\mathcal{L}$	binary cross-entropy of one entry; loss of one run (6.2)
$\bar{f}_k,\ f_{ik},\ f_{ik}^s,\ s^\star$	base rate; site frequency; per-storm frequency; nearest training storm (6.3)
$\text{BCE}_k,\ \hat{C}_k,\ C_k$	log loss; predicted and simulated counts (7.1–7.2)
$\text{Bias}_k,\ \text{WAPE}_k$	count bias and count error (7.2)
AP, AUC	ranking scores (7.3)
$w_{ij},\ j_i^\star,\ \text{ECE}_k$	window masses; true window; calibration error (7.5–7.6)

9 Assumptions worth knowing

1. Failure is permanent in both the simulator and the labels; recovery is not modelled.
2. Cascade failure times are resolved on the evaluation grid, not continuously.
3. Direct flood failures are independent across sites given the depth map; buffers are independent across links.
4. The fast model conditions on the realised seeds, so its target averages over the buffers only.
5. The elevation and flood-depth attributes of each site are placeholder constants in the current network file; the flood reaches the fast model only through the seed bit, the timing inputs and the storm summary.
6. The fast model is not given link attributes such as the buffer medians; it sees which links exist

and their relation type.

7. The flood model that produces the depth maps is outside this document.

Where the numbers come from. Hazus-type fragility curves: FEMA, Hazus Flood Model Technical Manual. Subway overload rule: Motter and Lai, Physical Review E 66, 065102, 2002. Attention message passing: Shi et al., "Masked label prediction", IJCAI 2021, as implemented in PyTorch Geometric (Fey and Lenssen, 2019). Hazard read-out: discrete-time survival analysis. Calibration error: Guo et al., ICML 2017. Optimiser: Kingma and Ba, "Adam", ICLR 2015.